



XUXEMONS

DOCUMENTACIÓ TÈCNICA

DE L'APLICACIÓ WEB
XUXEMON

Documentació completa sobre
l'arquitectura, desenvolupament
i funcionament de l'aplicació web
Xuxemon.



EQUIP DE DESENVOLUPAMENT

Anas • Kenneth • David



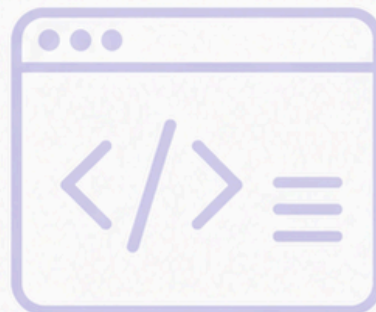
PROJECTE

Xuxemon – Aplicació Web



DATA

1 de Maig de 2026



ÍNDEX

1. Descripció general del projecte i objectius.....	3
2. Arquitectura de l'aplicació.....	3
2.1 Frontend.....	4
2.2 Backend.....	4
2.3 Base de dades.....	4
3. Diagrama de contenidors (Docker).....	5
4. Estructura de carpetes del projecte.....	5
4.1 Estructura general.....	6
4.2 Estructura principal del backend.....	7
4.3 Estructura principal del frontend.....	8
5. Tecnologies utilitzades i justificació.....	9
6. Requisits per al desplegament.....	10
7. Procés d'instal·lació i posada en marxa en local.....	10
7.1 Execució amb Docker Compose.....	10
7.2 Execució local sense Docker.....	10
8. Variables d'entorn utilitzades i la seva funció.....	11
8.1 Variables específiques del docker-compose.....	14
8.2 Observació sobre el frontend.....	14
9. Conclusions tècniques.....	14
10. DESPLEGAMENT DOCKER.....	15
10.1. Explicació del docker-compose.yml.....	15
DOCKERFILE CONTENIDOR BACKEND:.....	19
DOCKERFILE CONTENIDOR FRONTEND:.....	20
10.2 Comandes bàsiques d'ús (start, stop, rebuild).....	20
10.3 Captures i proves de funcionament del desplegament.....	21

Projecte: Xuxemon

Tipus de document: Documentació tècnica general

1. Descripció general del projecte i objectius

Xuxemon és una web orientada a la gestió i interacció amb animals virtuals anomenats Xuxemons. El projecte combina funcionalitats de registre i autenticació d'usuaris, perfil, inventari o motxilla, visualització de Xuxemons, configuració de joc i una capa social amb amistats i sol·licituds. L'aplicació s'ha desenvolupat amb una arquitectura separada entre frontend i backend, amb persistència de dades en base de dades relacional i suport de desplegament mitjançant contenidors.

Els objectius principals del projecte són els següents:

- Proporcionar una interfície web perquè l'usuari pugui registrar-se, iniciar sessió i gestionar el seu perfil.

- Permetre la visualització i administració dels Xuxemons disponibles i dels Xuxemons associats a cada usuari.

- Gestionar inventari, objectes i configuracions relacionades amb la lògica del joc.

- Incorporar funcionalitats socials com cerca d'usuaris, enviament de sol·licituds, acceptació o rebuig i llista d'amics.

- Separar clarament la capa de presentació, la lògica de negoci i la persistència de dades.

- Facilitar l'execució en entorn local i el desplegament amb Docker.

2. Arquitectura de l'aplicació

El projecte segueix una arquitectura de tipus client-servidor dividida en tres blocs principals: frontend, backend i base de dades. El frontend ofereix la interfície d'usuari i es comunica amb el backend mitjançant peticions HTTP a una API REST. El backend centralitza la lògica de negoci, l'autenticació, les validacions i l'accés a dades. La base de dades emmagatzema tota la informació persistent del sistema.

2.1 Frontend

El frontend està desenvolupat amb Angular i organitzat per components, serveis i rutes. Gestiona les pantalles principals de l'aplicació, com login, registre, home, Xuxedex, perfil, motxilla, panell d'administració i amistats. També inclou Guards de ruta per protegir l'accés a les pantalles d'usuaris autenticats i d'administradors.

La comunicació amb la API es fa principalment des dels serveis Angular, com ara **auth.service.ts**, **xuxemon.service.ts**, **friends.service.ts**, **game-config.service.ts** i **inventory.service.ts**.

2.2 Backend

El backend està desenvolupat amb Laravel. Implementa una API REST que dona servei a totes les funcionalitats del frontend. La lògica es distribueix principalment en controladors, models, middleware i serveis.

AuthenticatorController: login, registre i informació de l'usuari autenticat.

UserController: perfil, actualització de dades, desactivació i cerca d'usuaris.

XuxemonController: consulta i gestió de Xuxemons.

FriendRequestController: enviament, consulta i gestió de sol·licituds d'amistat.

FriendController: llistat i eliminació d'amics.

AdminController i **ConfigController:** funcionalitats de gestió i configuració.

L'autenticació de la API es basa en JWT, amb middleware específic per validar el token, comprovar si el compte està activa i restringir determinades rutes al rol d'administrador.

2.3 Base de dades

La base de dades principal és MySQL 8 en l'entorn Docker. En el backend també existeix un fitxer **database.sqlite**, útil com a suport local o durant certes fases de desenvolupament, però la configuració Docker del projecte està orientada a MySQL.

Les taules i entitats més rellevants del projecte són:

users: usuaris del sistema.

xuxemons: catàleg de criatures.

user_xuxemons: relació entre usuaris i Xuxemons.

items i **user_items:** objectes del joc i relació amb usuaris.

mochilas: inventari o motxilla d'usuari.

configs: configuracions globals del joc.

friend_requests: sol·licituds d'amistat amb estat pendent, acceptat o rebutjat.

friends i **amigos:** relacions d'amistat per compatibilitat funcional.

3. Diagrama de contenidors (Docker)

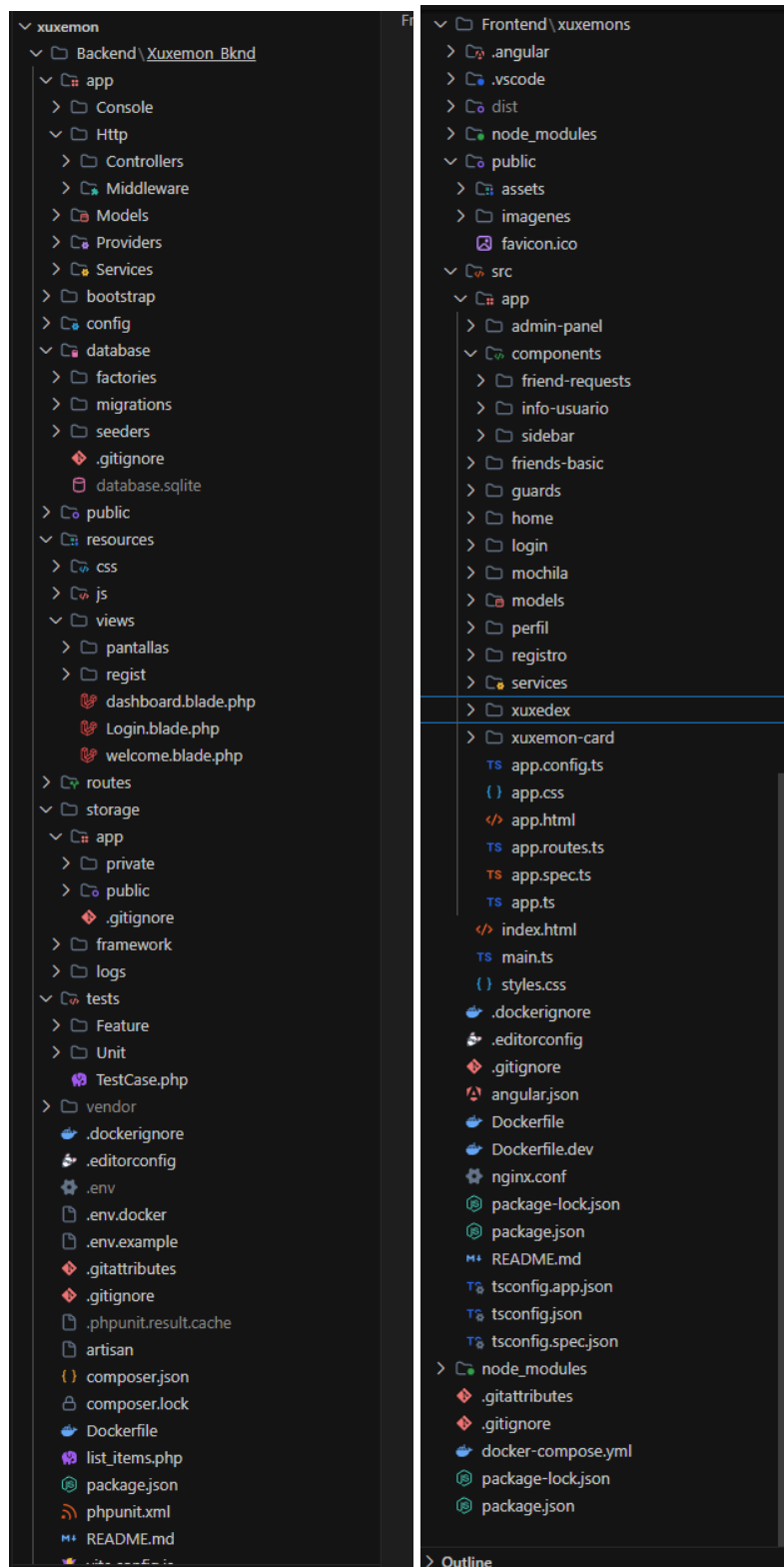
El desplegament amb Docker Compose defineix tres contenidors principals: base de dades MySQL, backend Laravel i frontend Angular. El backend depèn de la base de dades, i el frontend depèn del backend.

☐			xuxemon	-	-	-
☐			frontend	75db364dd8ac	xuxemon-frontend	4200:4200
☐			backend	9a4bfb3616b7	xuxemon-backend	8000:80
☐			bd	f1ab0be50104	mysql:8.0	3307:3306

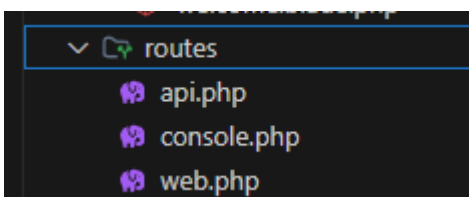
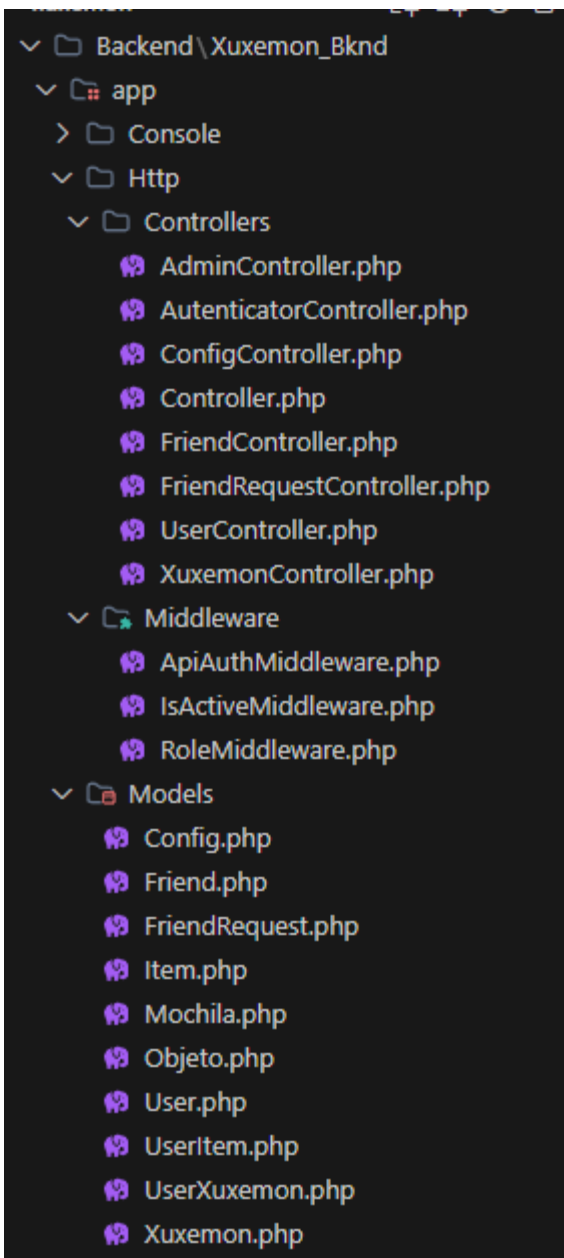
4. Estructura de carpetes del projecte

El repositori està organitzat en dues carpetes principals, una per al backend i una altra per al frontend, a més d'un fitxer **docker-compose.yml** en l'arrel del projecte.

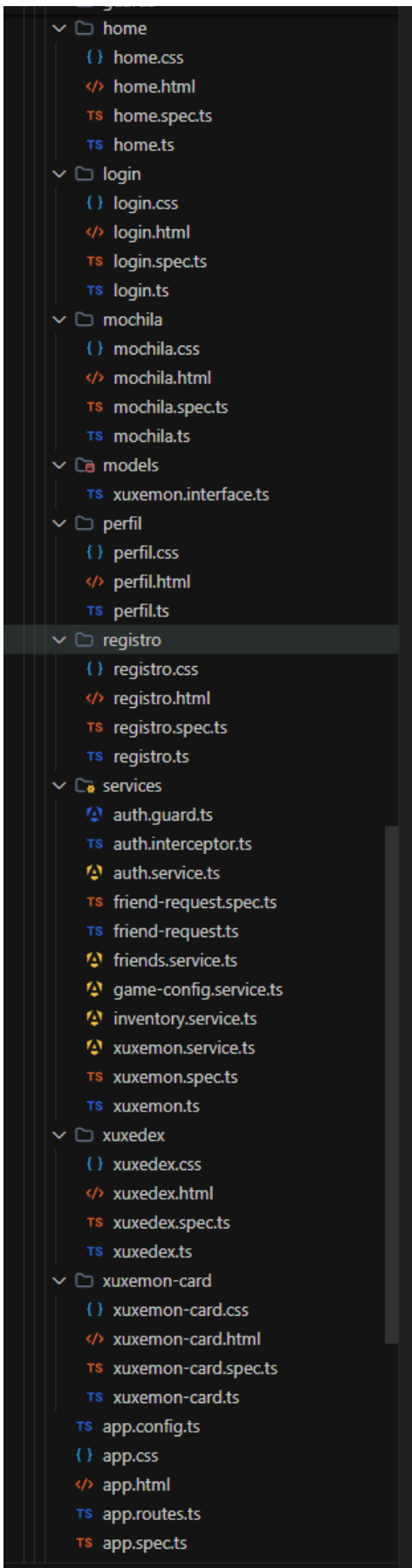
4.1 Estructura general



4.2 Estructura principal del backend



4.3 Estructura principal del frontend



5. Tecnologies utilitzades i justificació

Tecnologia	Ús en el projecte	Justificació
Angular	Desenvolupament del frontend SPA	Permet crear una interfície basada en components, amb routing, formularis i serveis HTTP.
TypeScript	Lògica del frontend	Aporta tipatge estàtic i una millor mantenibilitat del codi.

Tecnologia	Ús en el projecte	Justificació
RxJS	Fluxos reactius i estat compartit	S'utilitza per treballar amb observables i BehaviorSubject en diversos serveis de l'aplicació.
Bootstrap i Bootstrap Icons	Estils i components visuals	Faciliten un desenvolupament ràpid d'interfícies i una maquetació coherent.
Laravel	Backend i API REST	Framework robust per estructurar rutes, controladors, models, middleware, validacions i migracions.
PHP	Execució del backend	Versió compatible amb Laravel i amb bon rendiment per a la lògica.
MySQL	Persistència de dades	Base de dades relacional adequada per gestionar usuaris, inventari, configuracions i amistats.
JWT Auth	Autenticació de la API	Permet protegir rutes i identificar l'usuari autenticat de forma stateless.
Docker i Docker Compose	Contenidors i desplegament local	Faciliten una execució homogènia de frontend, backend i base de dades en qualsevol entorn.

6. Requisits per al desplegament

Docker Desktop o equivalent amb suport per Docker Compose.

Ports lliures: **4200** per al frontend, **8000** per al backend i **3307** per a MySQL.

Espai suficient en disc per a Node, Composer i volum persistent de MySQL.

Navegador web per provar l'aplicació web.

7. Procés d'instal·lació i posada en marxa en local

7.1 Execució amb Docker Compose

1. Clonar o descarregar el repositori del projecte.
2. Situar-se en l'arrel del projecte, on es troba el fitxer **docker-compose.yml**.
3. Comprovar que existeix el fitxer **Backend/Xuxemon_Bknd/.env.docker**.
4. Executar l'ordre **docker compose up --build**.
5. Esperar que els tres serveis s'iniciïn correctament.
6. Obrir el frontend a **http://localhost:4200**.
7. Accedir al backend a **http://localhost:8000** i a la API a **http://localhost:8000/api**.

7.2 Execució local sense Docker

Backend Laravel

1. Entrar a la carpeta **Backend/Xuxemon_Bknd**.
2. Instal·lar dependències amb **composer install**.
3. Crear el fitxer d'entorn a partir de **.env.example** si cal.
4. Generar la clau de Laravel amb **php artisan key:generate**.
5. Configurar les dades de connexió a la base de dades al fitxer **.env**.
6. Executar les migracions amb **php artisan migrate**.
7. Iniciar el servidor amb **php artisan serve**.

Frontend Angular

1. Entrar a la carpeta **Frontend/xuxemons**.
2. Instal·lar dependències amb **npm install**.
3. Executar **npm start** o **ng serve**.
4. Obrir el navegador a **http://localhost:4200**.

Cal tenir en compte que el frontend actualment apunta a **http://localhost:8000/api** amb URL escrita directament en diversos serveis. Si el backend s'executa en una altra adreça o port, caldria actualitzar aquestes rutes manualment o externalitzar-les a una configuració d'entorn.

8. Variables d'entorn utilitzades i la seva funció

El backend fa ús de variables d'entorn definides principalment en **.env**, **.env.example** i **.env.docker**. El frontend, en l'estat actual del projecte, no utilitza un sistema propi de variables d'entorn; la URL base de la API està declarada directament en diversos serveis Angular.

Variable	Funció
APP_NAME	Nom de l'aplicació Laravel.
APP_ENV	Defineix l'entorn d'execució, per exemple local.
APP_KEY	Clau interna de Laravel per xifrat i seguretat.
APP_DEBUG	Activa o desactiva el mode de depuració.
APP_URL	URL base del backend.
APP_LOCALE	Idioma principal de l'aplicació.

APP_FALLBACK_LOCALE	Idioma alternatiu si no hi ha traducció disponible.
APP_FAKER_LOCALE	Configuració regional per a dades de prova amb Faker.
BCRYPT_ROUNDS	Cost de l'algorisme de xifrat de contrasenyes.
LOG_CHANNEL, LOG_STACK, LOG_LEVEL	Controlen el sistema de logs de Laravel.
DB_CONNECTION	Tipus de base de dades a utilitzar, en aquest projecte MySQL.
DB_HOST	Host del servidor de base de dades. En Docker és bd .
DB_PORT	Port de connexió a la base de dades.

Variable	Funció
DB_DATABASE	Nom de la base de dades.
DB_USERNAME	Usuari de connexió a la base de dades.
DB_PASSWORD	Contrasenya de connexió a la base de dades.
SESSION_DRIVER	Sistema d'emmagatzematge de sessions. En Docker es fa servir file .
SESSION_LIFETIME	Durada de la sessió en minuts.

SESSION_ENCRYPT, SESSION_PATH, SESSION_DOMAIN	Configuració del comportament de les sessions.
BROADCAST_CONNECTION	Defineix el sistema de broadcasting.
FILESYSTEM_DISK	Disc d'emmagatzematge per defecte.
QUEUE_CONNECTION	Defineix el sistema de cues del backend.
CACHE_STORE	Tipus d'emmagatzematge per a la memòria cau.
MEMCACHED_HOST	Host de Memcached si s'utilitza.
REDIS_CLIENT, REDIS_HOST, REDIS_PASSWORD, REDIS_PORT	Configuració de Redis si s'empra com a suport de cache o sessions.
MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM_ADDRESS, MAIL_FROM_NAME	Configuració del servei de correu electrònic.

Variable	Funció
AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_DEFAULT_REGION, AWS_BUCKET, AWS_USE_PATH_STYLE_ENDPOINT	Variables reservades per a integració amb serveis AWS si es necessiten.
VITE_APP_NAME	Nom de l'aplicació exposat a l'entorn de compilació frontend del backend.

JWT_SECRET	Secret utilitzat per signar i validar els tokens JWT de l'autenticació.
------------	---

8.1 Variables específiques del docker-compose

MYSQL_ROOT_PASSWORD: contrasenya de l'usuari root de MySQL.

MYSQL_DATABASE: base de dades que es crea automàticament a l'inici.

MYSQL_USER: usuari específic per a l'aplicació.

MYSQL_PASSWORD: contrasenya d'aquest usuari de l'aplicació.

8.2 Observació sobre el frontend

Actualment, el frontend no fa servir fitxers d'entorn propis per a la configuració de la URL de la API. Aquesta ruta està definida manualment a diversos serveis Angular com **auth.service.ts**, **friends.service.ts**, **xuxemon.service.ts** i **game-config.service.ts**. Des del punt de vista tècnic, seria recomanable una millora futura per centralitzar aquesta configuració en fitxers d'entorn d'Angular segons l'entorn de desenvolupament o producció.

9. Conclusions tècniques

Xuxemon és una web construïda amb Angular al frontend, Laravel al backend i MySQL com a base de dades principal. El projecte està preparat per executar-se en local mitjançant Docker Compose i també admet una posada en marxa manual. La seva estructura modular facilita l'ampliació funcional, especialment en àrees com autenticació, inventari, gestió de Xuxemons i sistema social.

A nivell tècnic, els punts forts més destacables són la separació de responsabilitats, l'ús d'API REST, el model de contenidors per al desplegament i l'ús de serveis reactius al frontend. Com a aspecte millorable, destaca la conveniència de centralitzar la configuració del frontend amb variables d'entorn pròpies per evitar dependències de URLs escrites directament al codi.

10. DESPLEGAMENT DOCKER

10.1. Explicació del docker-compose.yml

Per fer aquest projecte hem utilitzat 3 contenidors un que guarda la part de frontend, l'altre guarda el backend de la web i per ultim un contenidor que guarda la base de dades on es guarda la informació que gestiona el contenedor del backend.

- bd (MySQL 8.0)
- backend (Laravel)
- frontend (Angular)

```
docker-compose.yml X
docker-compose.yml
1  services:
2    bd:
3      image: mysql:8.0
4      container_name: xuxemon-bd
5      environment:
6        MYSQL_ROOT_PASSWORD: root
7        MYSQL_DATABASE: xuxemon
8        MYSQL_USER: xuxemon_user
9        MYSQL_PASSWORD: xuxemon_pass
10     ports:
11       - "3307:3306"
12     volumes:
13       - mysql-data:/var/lib/mysql
14     healthcheck:
15       test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
16       interval: 10s
17       timeout: 5s
18       retries: 10
19
20     backend:
21       build: ./Backend/Xuxemon_Bknd
22       container_name: xuxemon-backend
23       ports:
24         - "8000:80"
25       depends_on:
26         bd:
27           condition: service_healthy
28       env_file:
29         - ./Backend/Xuxemon_Bknd/.env.docker
30       volumes:
31         - ./Backend/Xuxemon_Bknd:/var/www/html
32
33     frontend:
34       build:
35         context: ./Frontend/xuxemons
36         dockerfile: Dockerfile.dev
37       container_name: xuxemon-frontend
38       ports:
39         - "4200:4200"
40       depends_on:
41         - backend
42       volumes:
43         - ./Frontend/xuxemons:/app
44         - /app/node_modules
45
46     volumes:
47       mysql-data:
48
```

Servei bd:

- Imatge: mysql:8.0
- Contenidor: xuxemon-bd
- Ports: 3307:3306
- Variables d'entorn: MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD
- Volum persistent: mysql-data:/var/lib/mysql
- Healthcheck: mysqladmin ping per validar que la BD estigui operativa.

```
bd:
  image: mysql:8.0
  container_name: xuxemon-bd
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: xuxemon
    MYSQL_USER: xuxemon_user
    MYSQL_PASSWORD: xuxemon_pass
  ports:
    - "3307:3306"
  volumes:
    - mysql-data:/var/lib/mysql
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-uroot", "-proot"]
    interval: 10s
    timeout: 5s
    retries: 10
```

Servei backend:

- Build local des de ./Backend/Xuxemon_Bknd (Dockerfile propi)
- Contenidor: xuxemon-backend
- Ports: 8000:80
- Dependencia: espera a bd amb condition: service_healthy
- Entorn: es carrega des de Backend/Xuxemon_Bknd/.env.docker
- Volum bind: ./Backend/Xuxemon_Bknd:/var/www/html (hot-reload de codi)

```
backend:
  build: ./Backend/Xuxemon_Bknd
  container_name: xuxemon-backend
  ports:
    - "8000:80"
  depends_on:
    bd:
      condition: service_healthy
  env_file:
    - ./Backend/Xuxemon_Bknd/.env.docker
  volumes:
    - ./Backend/Xuxemon_Bknd:/var/www/html
```

Servei frontend:

- Build local des de ./Frontend/xuxemons (Dockerfile.dev)
- Contenidor: xuxemon-frontend
- Ports: 4200:4200
- Dependencia: backend
- Volumes:
 - * ./Frontend/xuxemons:/app
 - * /app/node_modules (volum separat per evitar conflictes amb node_modules local)

```
frontend:
  build:
    context: ./Frontend/xuxemons
    dockerfile: Dockerfile.dev
  container_name: xuxemon-frontend
  ports:
    - "4200:4200"
  depends_on:
    - backend
  volumes:
    - ./Frontend/xuxemons:/app
    - /app/node_modules
```

Volumes:

- mysql-data: guarda les dades de MySQL de forma persistent.

```
volumes:
  mysql-data:
```

-Frontend:

```
volumes:
  - ./Frontend/xuxemons:/app
  - /app/node_modules
```

-Backend:

```
volumes:  
  - ./Backend/Xuxemon_Bknd:/var/www/html
```

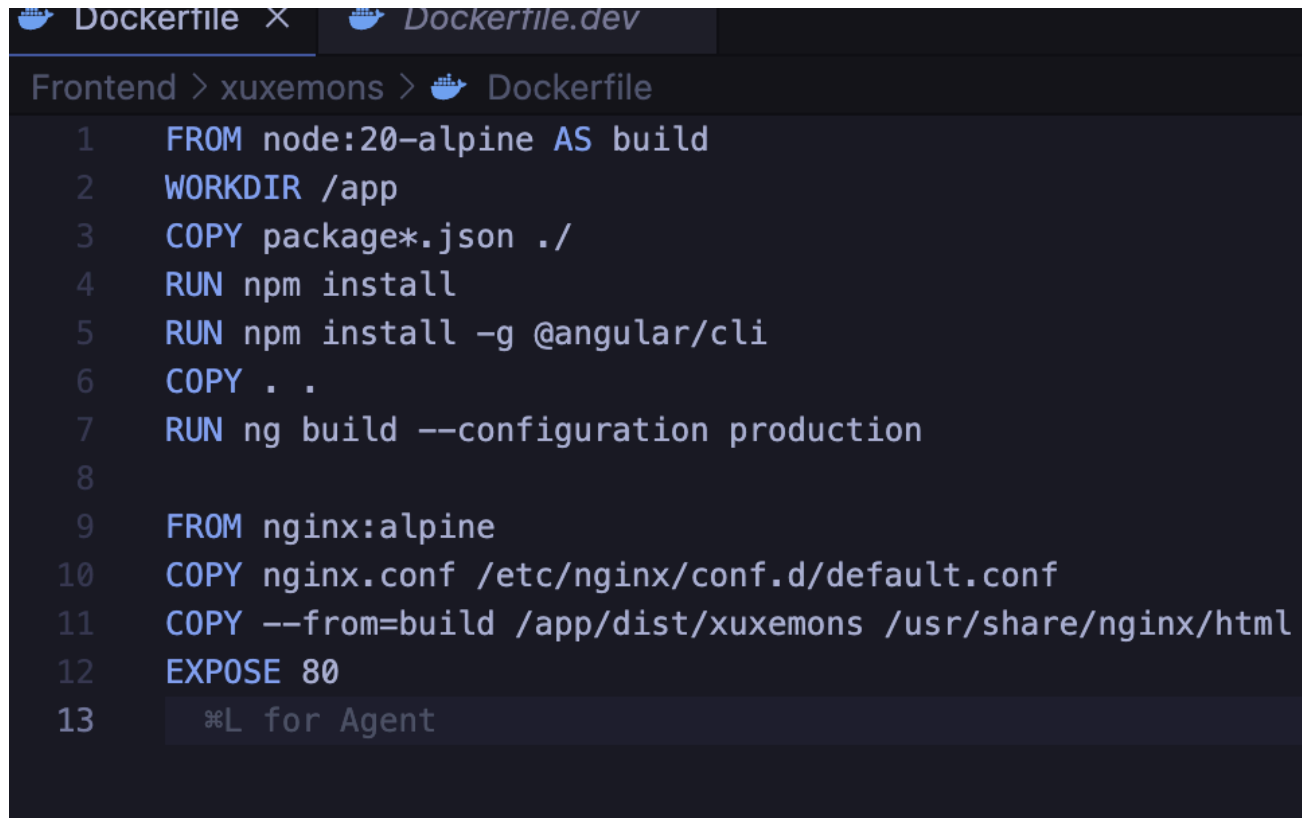
Xarxa:

- Compose crea una xarxa interna per defecte (xuxemon_default) i permet comunicacio entre serveis per nom (ex: DB_HOST=bd).

DOCKERFILE CONTENIDOR BACKEND:

```
Backend > Xuxemon_Bknd > Dockerfile  
1 FROM php:8.2-apache  
2  
3 RUN apt-get update && apt-get install -y \  
4     git \  
5     zip \  
6     unzip \  
7     libzip-dev \  
8     && docker-php-ext-install pdo_mysql zip  
9  
10 RUN a2enmod rewrite  
11  
12 ENV APACHE_DOCUMENT_ROOT=/var/www/html/public  
13 RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/*.conf  
14  
15 COPY --from=composer:latest /usr/bin/composer /usr/bin/composer  
16  
17 WORKDIR /var/www/html  
18 COPY . .  
19  
20 RUN composer install --no-dev --no-interaction --prefer-dist --optimize-autoloader  
21  
22 RUN chown -R www-data:www-data storage bootstrap/cache  
23  
24 EXPOSE 80
```

DOCKERFILE CONTENIDOR FRONTEND:



```
1 FROM node:20-alpine AS build
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 RUN npm install -g @angular/cli
6 COPY . .
7 RUN ng build --configuration production
8
9 FROM nginx:alpine
10 COPY nginx.conf /etc/nginx/conf.d/default.conf
11 COPY --from=build /app/dist/xuxemons /usr/share/nginx/html
12 EXPOSE 80
13 %%L for Agent
```

10.2 Comandes bàsiques d'ús

Start (arrencar stack):

```
docker compose up -d
```

Stop (aturar contenidors, mantenint-los):

```
docker compose stop
```

Restart contenidors aturats:

```
docker compose start
```

Down (aturar i eliminar contenidors i xarxa del projecte):

`docker compose down`

Rebuild basic (recrear imatges i arrencar):

`docker compose up -d --build`

Rebuild complet (neteja + build des de zero):

`docker compose down`

`docker compose build --no-cache`

`docker compose up -d`

Neteja total incloent volums (esborra dades MySQL):

`docker compose down -v`

Comprovacions utils:

`docker compose ps`

`docker compose logs -f`

`docker compose logs -f bd`

`docker compose logs -f backend`

`docker compose logs -f frontend`

10.3 Captures i proves de funcionament del desplegament

3.1 Versió de Docker Compose

Comanda:

`docker compose version`

Sortida:

Docker Compose version v5.1.3

3.2 Serveis detectats per Compose

Comanda:

```
docker compose config --services
```

Sortida:

```
bd
backend
frontend
```

3.3 Estat actual del stack

Comanda:

```
docker compose ps
```

Sortida:

```
anasmbk@Mac xuxemon % docker compose ps
NAME                IMAGE              COMMAND              SERVICE    CREATED        STATUS              PORTS
xuxemon-backend     xuxemon-backend   "docker-php-entryp... backend     10 hours ago   Up 10 hours        0.0.0.0:8000->80/tcp, [::]:8000->80/tcp
xuxemon-bd          mysql:8.0         "docker-entrypoint.s... bd         10 hours ago   Up 10 hours (healthy) 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
xuxemon-frontend    xuxemon-frontend  "docker-entrypoint.s... frontend   10 hours ago   Up 10 hours        0.0.0.0:4200->4200/tcp, [::]:4200->4200/tcp
anasmbk@Mac xuxemon %
```

3.4 Prova de proxy de Docker Desktop

Comanda:

```
docker info | grep -Ei 'HTTP Proxy|HTTPS Proxy|No Proxy'
```

Sortida:

```
anasmbk@Mac xuxemon % docker info | grep -Ei 'HTTP Proxy|HTTPS Proxy|No Proxy'

HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
anasmbk@Mac xuxemon % |
```


3.5 Prova de connectivitat al host de blobs de Docker

Comanda:

```
curl -I --max-time 20 https://docker-images-prod.6aa30f8b08e16409b46e0173d6de2f56.r2.cloudflarestorage.com
```

Sortida:

☐	▼		xuxemon	-	-	-	5.98%	10 hours ago	
☐			bd	e9cad58951bd 	mysql:8.0	3307:3306 	1.15%	10 hours ago	
☐			backend	c3b56699a91d	xuxemon-backend	8000:80 	0.01%	10 hours ago	
☐			frontend	cdf4243517f	xuxemon-frontend	4200:4200 	4.82%	10 hours ago	